

AI Use Disclosure

Assignment: Extracting IPv4 Addresses from Noisy Text

AI Tool Used: Claude (Anthropic) 5.0, web/chat interface

1. Summary of AI Involvement

Claude was used within this project to generate a C program that extracts and validates a single IPv4 address from a text string provided by the user. Throughout this project, I utilized Claude three times and after each iteration I reviewed and, if necessary, modified the code before committing the changes to my GitHub repository.

Below is a complete record of each prompt I gave Claude, what it produced, the changes I made in response, and how I verified the results.

2. Prompt by Prompt Log

Prompt 1 – Initial Request

Prompt:

“Write a C program that extracts and validates a singular IPv4 address from a text string.

Functions needed:

- `int main()`
- `int extractIPv4(const char* str, unsigned long* outAddress, int* outPort)`

The function main consists of a loop that prompts to input a text string like the following:

“Enter a string (or 'END' to quit):” . Then, `extractIPv4` is called like the following:

`extractIPv4(text, textLength, *outAddress, *outPort)`. When `extractIPv4` returns a 1, `*outAddress` holds the IPv4 address and `*outPort` holds –1 if not port was provided and otherwise the port value. On success, the following is printed: “Extracted IPv4 address: A.B.C.D (decimal value: <decimalValue>, port: <portValue>)”. When the user inputs “END”, “Program terminated” is printed and loop is terminated.

The function `extractIPv4` returns a 1 if a valid address was found and 0 otherwise. The function performs digit accumulation to find the first valid address. A valid IPv4 address consists of four octets that each consist of 1-3 digits (values 0-255) and there are only

leading zeros if the value is 0. An optional :port may follow only the fourth octet and consists of exactly 1-5 digits (values 0-65535) and there are only leading zeros if the value is 0. A correct IPv4 address looks like the following: 137.62.0.19:419. An incorrect IPv4 address looks like the following: 132.4.12. If the address is not fully valid, it is rejected. When an address followed by a colon (indicating the start of a port) is found, if either the address or port are invalid, both the address and port are rejected. When the first valid address is found, the address is stored in `*outAddress`, the port is stored in `*outPort` (0-65535 if a port is provided and -1 otherwise), and the function returns 1, without needing to look at the rest of the text string.”

AI Output:

Claude produced a complete `extract_ipv4.c` file containing an `extractIPv4()` function that scans the input text for the first valid IPv4 address, and a `main()` function implementing the sentinel-controlled input loop.

My Modifications and Verification:

I updated the main function to display “none” instead of “-1” for a missing port, and changed the final else branch’s message to “Invalid input: no valid IPv4 address found” in place of “No valid IPv4 address found.”

I also identified a bug, in which the code did not correctly invalidate an IPv4 address when a stray period or colon appeared directly adjacent to an otherwise valid address. I added the following check to fix this issue:

```
else if (str[pos] == '.')
{
    success = 0;
    break;
}
```

These changes are recorded in the second commit of my GitHub repository.

To verify the changes, I read through the code line by line to confirm I understood the logic and to check for errors, and I ran the compiled program against several edge cases through the sentinel input loop.

Prompt 2 – Create Test Cases

Prompt:

“Attachment: extract_ipv4.c

The file extract_ipv4.c contains my modifications to the C file you created.

The following modifications were made:

1. extractIPv4 function was updated to invalidate an address if there is a trailing ‘.’. For example, “octet.octet.octet.octet.” is invalid.
2. main function was updated to display “port: none” when –1 is provided for a port and the invalid input print statement was changed.

Updates to be added to this file are the following:

1. Create test cases that test the following edge cases and others you see fit to test:
 - a. Invalid octet range.
 - b. Invalid port range.
 - c. Trailing ‘.’ or trailing ‘:’ which are invalid.
 - d. Invalid number of octets.
 - e. A port being placed in the middle of the octets. Ex:
<octet.octet:port.octet.octet>
 - f. Test that the function will return the first valid IPv4 address by having multiple.
 - g. Test that an invalid port won’t return anything even if the IPv4 address is correct.
 - h. Test that something like the following will evaluate as invalid: “connecting to 1234.12.12.12”.

AI Output:

Claude added various unit tests to [extractipv4.c](#). These unit tests were connected within `main()` to be run by a `--test` command line flag. Thus, by running the executable with `--test` (ex. `./581hw1 --test`) the automated tests are run without affecting a user’s normal interactive use. In the process, Claude also identified a bug where a valid IPv4 address occurring later in the input would not be detected if it followed by an invalid address of the form "octet.octet.octet.octet." due to a break statement in the code I had added in Prompt 1:

```
else if (str[pos] == '.')  
{
```

```
        success = 0;

        break;
    }
```

My Modifications and Verification:

I reviewed the issue Claude identified and determined that removing the break statement resolved the issue, resulting in the following:

```
        else if (str[pos] == '.')
        {
            success = 0;
        }
```

To verify Claude's changes, I again read through the code line by line to confirm I understood the logic and to check for any errors it might have introduced. Running the unit tests afterward showed that 22 of 23 test cases passed. The program failed the “too many octets test” by incorrectly accepting the first four octets of an address in the form octet.octet.octet.octet.octet instead of rejecting it entirely.

Prompt 3 – Modify Code to Pass All Tests

Prompt:

“Attachment: extract_ipv4.c

With those changes applied, the program fails the test case of Too many octets (5). The following is the fail message:

```
[FAIL] Too many octets (5) input: "1.2.3.4.5" expected: return=0 actual: return=1,
address=33752069, port=-1
```

Modify the code to ensure this test is passed.”

AI Output:

Claude modified the address scanning logic in [extract_ipv4.c](#) to make this test pass. One key change introduced was setting success to 0 whenever a . immediately follows a fully parsed four octet sequence. The second, more significant change was to skip past an entire malformed run of octets once it is found to be out of range, rather than re-scanning

from the next character:

```
int skip = i;
    while (skip < len && (isdigit((unsigned char)str[skip]) ||
str[skip] == '.'))
    {
        skip++;
    }
    i = skip;
    continue;
```

My Modifications and Verification:

To verify the change, I read through the code line by line to confirm I understood the logic and to check for any remaining errors. I re ran all the unit tests and confirmed all cases passed. I also further checked the program by using the sentinel input loop to confirm the logic held up beyond the automated tests. I also added my own comments to the code for clarity.

I have review and understand the final code in this submission, including all AI-assisted portions and I take full responsibility for the correctness of the program.

Signature: *Heidi Schieber*

Date: 09/23/2026